

Oracle PL/Sql

widoki, funkcje, procedury, triggerzy ćwiczenie

Imiona i nazwiska autorów : Michał Białas, Jakub Turek

Tabele

- **Trip** - wycieczki
 - **trip_id** - identyfikator, klucz główny
 - **trip_name** - nazwa wycieczki
 - **country** - nazwa kraju
 - **trip_date** - data
 - **max_no_places** - maksymalna liczba miejsc na wycieczkę
- **Person** - osoby
 - **person_id** - identyfikator, klucz główny
 - **firstname** - imię
 - **lastname** - nazwisko
- **Reservation** - rezerwacje/bilety na wycieczkę
 - **reservation_id** - identyfikator, klucz główny
 - **trip_id** - identyfikator wycieczki
 - **person_id** - identyfikator osoby
 - **status** - status rezerwacji
 - **N** – New - Nowa
 - **P** – Confirmed and Paid – Potwierdzona i zapłacona
 - **C** – Canceled - Anulowana
- **Log** - dziennik zmian statusów rezerwacji
 - **log_id** - identyfikator, klucz główny
 - **reservation_id** - identyfikator rezerwacji
 - **log_date** - data zmiany
 - **status** - status

```
create sequence s_person_seq
start with 1
increment by 1;

create table person
(
  person_id int not null
    constraint pk_person
      primary key,
  firstname varchar(50),
  lastname varchar(50)
)

alter table person
  modify person_id int default s_person_seq.nextval;
```

```
create sequence s_trip_seq
start with 1
increment by 1;

create table trip
(
  trip_id int not null
    constraint pk_trip
      primary key,
  trip_name varchar(100),
  country varchar(50),
  trip_date date,
  max_no_places int
);

alter table trip
  modify trip_id int default s_trip_seq.nextval;
```

```
create sequence s_reservation_seq
  start with 1
  increment by 1;

create table reservation
(
  reservation_id int not null
    constraint pk_reservation
      primary key,
  trip_id int,
  person_id int,
  status char(1)
);

alter table reservation
  modify reservation_id int default s_reservation_seq.nextval;

alter table reservation
add constraint reservation_fk1 foreign key
( person_id ) references person ( person_id );

alter table reservation
add constraint reservation_fk2 foreign key
( trip_id ) references trip ( trip_id );

alter table reservation
add constraint reservation_chk1 check
(status in ('N','P','C'));
```

```
create sequence s_log_seq
  start with 1
  increment by 1;

create table log
(
  log_id int not null
    constraint pk_log
      primary key,
  reservation_id int not null,
  log_date date not null,
  status char(1)
);

alter table log
  modify log_id int default s_log_seq.nextval;

alter table log
add constraint log_chk1 check
(status in ('N','P','C')) enable;

alter table log
add constraint log_fk1 foreign key
( reservation_id ) references reservation ( reservation_id );
```

Dane

Należy wypełnić tabele przykładowymi danymi

- 4 wycieczki
- 10 osób
- 10 rezerwacji

Dane testowe powinny być różnorodne (wycieczki w przyszłości, wycieczki w przeszłości, rezerwacje o różnym statusie itp.) tak, żeby umożliwić testowanie napisanych procedur.

W razie potrzeby należy zmodyfikować dane tak żeby przetestować różne przypadki.

```
-- trip
insert into trip(trip_name, country, trip_date, max_no_places)
values ('Wycieczka do Paryża', 'Francja', to_date('2023-09-12', 'YYYY-MM-DD'), 3);

insert into trip(trip_name, country, trip_date, max_no_places)
values ('Piękny Kraków', 'Polska', to_date('2025-05-03', 'YYYY-MM-DD'), 2);

insert into trip(trip_name, country, trip_date, max_no_places)
values ('Znow do Francji', 'Francja', to_date('2025-05-01', 'YYYY-MM-DD'), 2);

insert into trip(trip_name, country, trip_date, max_no_places)
```

```
values ('Hel', 'Polska', to_date('2025-05-01','YYYY-MM-DD'), 2);

insert into trip(trip_name, country, trip_date, max_no_places)
values ('Rio De Janeiro', 'Brazylia', to_date('2026-10-26','YYYY-MM-DD'), 6);

-- person
insert into person(firstname, lastname)
values ('Jan', 'Nowak');

insert into person(firstname, lastname)
values ('Jan', 'Kowalski');

insert into person(firstname, lastname)
values ('Jan', 'Nowakowski');

insert into person(firstname, lastname)
values ('Novak', 'Nowak');

insert into person(firstname, lastname)
values ('Kamil', 'Ślimak');

insert into person(firstname, lastname)
values ('Maja', 'Kania');

insert into person(firstname, lastname)
values ('Michał', 'Borek');

insert into person(firstname, lastname)
values ('Tomek', 'Worek');

insert into person(firstname, lastname)
values ('Olaf', 'Nowak');

insert into person(firstname, lastname)
values ('Marcin', 'Torcin');

-- reservation
-- trip1
insert into reservation(trip_id, person_id, status)
values (1, 1, 'P');

insert into reservation(trip_id, person_id, status)
values (1, 2, 'N');

-- trip 2
insert into reservation(trip_id, person_id, status)
values (2, 1, 'P');

insert into reservation(trip_id, person_id, status)
values (2, 4, 'C');

-- trip 3
insert into reservation(trip_id, person_id, status)
values (3, 4, 'P');

insert into reservation(trip_id, person_id, status)
values (3, 5, 'C');

insert into reservation(trip_id, person_id, status)
values (3, 8, 'N');

--trip 4
insert into reservation(trip_id, person_id, status)
values (4, 10, 'C');

insert into reservation(trip_id, person_id, status)
values (4, 9, 'P');

-- trip 5
insert into reservation(trip_id, person_id, status)
values (5, 6, 'P');

insert into reservation(trip_id, person_id, status)
values (5, 7, 'P');
```

proszę pamiętać o zatwierdzeniu transakcji

Zadanie 0 - modyfikacja danych, transakcje

Należy przeprowadzić kilka eksperymentów związanych ze wstawianiem, modyfikacją i usuwaniem danych oraz wykorzystaniem transakcji

Skomentuj działanie transakcji. Jak działa polecenie `commit`, `rollback`? Co się dzieje w przypadku wystąpienia błędów podczas wykonywania transakcji? Porównaj sposób programowania operacji wykorzystujących transakcje w Oracle PL/SQL ze znanym ci systemem/językiem MS Sqlserver T-SQL

pomocne mogą być materiały dostępne tu: <https://upel.agh.edu.pl/mod/folder/view.php?id=411834> w szczególności dokumenty: [10_modyf_ora_north.pdf](#), [20_ora_plsql_north.pdf](#)

```
-- przykłady, kod, zrzuty ekranów, komentarz ...
-- transakcja dla MS Sqlserver T-SQL
-- zachowując zasady ACID w momencie błędu w transakcji wykonuje się ROLLBACK i wszystkie update-y się cofają
BEGIN;

INSERT INTO reservation (reservation_id, trip_id, person_id, status)
VALUES (1, 10, 5);

INSERT INTO log (log_id, reservation_id, log_date, status)
VALUES (1, 1, CURRENT_DATE, 'N');

COMMIT;

-- Dla Oracle nie piszemy BEGIN on sam wykrywa początek transakcji i jedynie COMMIT na koncu mówi o wykonaniu całości transakcji np:

UPDATE person SET lastname = lastname + 'Kowalski' WHERE person_id = 5

UPDATE person SET lastname = lastname + 'Kowalska' WHERE person_id = 10

COMMIT

-- W przypadku ROLLBACKOW jeśli przykładowo tworzymy nową tabelę Busses to w Oracle po stworzeniu jej i wykonaniu ROLLBACK nie zmieni nic -
tabela powstanie a rollback się nie wykona ponieważ COMMIT zrobi się automatycznie

create table Busses
(
    bus_id int not null
        constraint pk_bus
        primary key,
    bus_size int not null,
);
rollback

-- dla MS Sqlserver T-SQL powstaje tabela a po wykonaniu rollbacku zostaje ona usunięta
begin
create table Busses
(
    bus_id int not null
        constraint pk_bus
        primary key,
    bus_size int not null,
);
rollback

-- dla Oracle po pojawieniu się błędu pomimo tego dalej mamy możliwość wykonania rollbacku albo commita

INSERT INTO reservation VALUES (1, 10, 5, 'N');

-- błąd (np. zły typ danych)
INSERT INTO reservation VALUES ('abc', 10, 5, 'N');

COMMIT;

-- dla T-SQL błąd zatrzymuje instrukcję ale transakcja dalej istnieje
BEGIN TRAN;

INSERT INTO reservation VALUES (1, 10, 5, 'N');

-- błąd
INSERT INTO reservation VALUES ('abc', 10, 5, 'N');

COMMIT;
```

Zadanie 1 - widoki

Tworzenie widoków. Należy przygotować kilka widoków ułatwiających dostęp do danych. Należy zwrócić uwagę na strukturę kodu (należy unikać powielania kodu)

Widoki:

- `vw_reservation`
 - widok łączy dane z tabel: `trip`, `person`, `reservation`
 - zwracane dane: `reservation_id`, `country`, `trip_date`, `trip_name`, `firstname`, `lastname`, `status`, `trip_id`, `person_id`
- `vw_trip`
 - widok pokazuje liczbę wolnych miejsc na każdą wycieczkę

- zwracane dane: `trip_id`, `country`, `trip_date`, `trip_name`, `max_no_places`, `no_available_places` (liczba wolnych miejsc)
- `vw_available_trip`
 - podobnie jak w poprzednim punkcie, z tym że widok pokazuje jedynie dostępne wycieczki (takie które są w przyszłości i są na nie wolne miejsca)

Proponowany zestaw widoków można rozbudować wedle uznania/potrzeb

- np. można dodać nowe/pomocnicze widoki, funkcje
- np. można zmienić def. widoków, dodając nowe/potrzebne pola

Zadanie 1 - rozwiązanie

```
-- wyniki, kod, zrzuty ekranów, komentarz ...

create view vw_reservation as
select r.reservation_id,
       t.country,
       t.trip_date,
       t.trip_name,
       p.firstname,
       p.lastname,
       r.status,
       t.trip_id,
       p.person_id
from reservation r
     inner join trip t on t.trip_id = r.trip_id
     inner join person p on r.person_id = p.person_id

create view vw_trip as
select t.trip_id,
       t.country,
       t.trip_date,
       t.trip_name,
       t.max_no_places,
       t.max_no_places - count(distinct r.person_id) as places_left
from trip t
     left join reservation r
           on t.trip_id = r.trip_id
           and not exists (select 1
                          from reservation r2
                          where r2.person_id = r.person_id
                          and r2.trip_id = r.trip_id
                          and r2.status = 'C')
group by t.trip_id,
         t.country,
         t.trip_date,
         t.trip_name,
         t.max_no_places;

create view vw_available_trip as
select * from vw_trip where vw_trip.trip_date >= current_date and vw_trip.places_left > 0
```

Zadanie 2 - funkcje

Tworzenie funkcji pobierających dane/tabele. Podobnie jak w poprzednim przykładzie należy przygotować kilka funkcji ułatwiających dostęp do danych

Procedury:

- `f_trip_participants`
 - zadaniem funkcji jest zwrócenie listy uczestników wskazanej wycieczki
 - parametry funkcji: `trip_id`
 - funkcja zwraca podobny zestaw danych jak widok `vw_reservation`
- `f_person_reservations`
 - zadaniem funkcji jest zwrócenie listy rezerwacji danej osoby
 - parametry funkcji: `person_id`
 - funkcja zwraca podobny zestaw danych jak widok `vw_reservation`
- `f_available_trips_to`
 - zadaniem funkcji jest zwrócenie listy wycieczek do wskazanego kraju, dostępnych w zadanym okresie czasu (od `date_from` do `date_to`)
 - dostępnych czyli takich na które są wolne miejsca
 - parametry funkcji: `country`, `date_from`, `date_to`

Funkcje powinny zwracać tabelę/zbiór wynikowy. Należy rozważyć dodanie kontroli parametrów, (np. jeśli parametrem jest `trip_id` to można sprawdzić czy taka wycieczka istnieje). Podobnie jak w przypadku widoków należy zwrócić uwagę na strukturę kodu

Czy kontrola parametrów w przypadku funkcji ma sens?

- jakie są zalety/wady takiego rozwiązania?

Proponowany zestaw funkcji można rozbudować wedle uznania/potrzeb

- np. można dodać nowe/pomocnicze funkcje/procedury

Zadanie 2 - rozwiązanie

```
-- wyniki, kod, zrzuty ekranów, komentarz ...
create or replace type participants as OBJECT
(
  reservation_id number,
  firstname varchar2(50),
  lastname varchar2(50),
  status char,
  person_id number
);

create or replace type participants_table is table of participants;

create function f_trip_participants (target_trip_id number)
return participants_table
as result participants_table;
  valid number;
begin
  select count(TRIP_ID) into valid from trip where trip_id = target_trip_id;

  if valid = 0 then
    raise_application_error(-20001, 'trip not found');
  end if;

  select participants(v.reservation_id, v.firstname, v.lastname, v.status, v.person_id)
  bulk collect
  into result
  from vw_reservation v where v.trip_id = target_trip_id;

  return result;
end;

create or replace type reservations_row as OBJECT
(
  reservation_id number,
  status char,
  trip_id number,
  trip_name varchar(50),
  country varchar2(50),
  trip_date date
);

create or replace type reservations_table is table of reservations_row;

create function f_person_reservations(target_person_id number)
return reservations_table as
  result reservations_table;
  valid number;
begin
  select count(person_id)
  into valid
  from person
  where person_id = target_person_id;

  if valid <= 0 then
    raise_application_error(-20001, 'no participant found');
  end if;

  select reservations_row(reservation_id,
    status,
    trip_id,
    trip_name,
    country,
    trip_date)
  bulk collect
  into result
  from vw_reservation
  where target_person_id = person_id;
  return result;
```

```

end;

create type trips_row as object
(
    trip_id number,
    trip_name varchar2(100),
    trip_date date
)

create type trips_table is table of trips_row

create function f_available_trips_to(a_country varchar2, a_date_from date, a_date_to date)
return trips_table as
    result trips_table;
    value number;
begin
    select count(*) into value from trip where a_country = country;

    if value = 0 then
        raise_application_error(-20001, 'country not found');
    end if;

    select trips_row(trip_id, trip_name, trip_date)
        bulk collect into result from vw_trip where country = a_country and places_left > 0
        and trip_date between a_date_from and a_date_to;
    return result;
end;

```

Zadanie 3 - procedury

Tworzenie procedur modyfikujących dane. Należy przygotować zestaw procedur pozwalających na modyfikację danych oraz kontrolę poprawności ich wprowadzania

Procedury

- **p_add_reservation**
 - zadaniem procedury jest dopisanie nowej rezerwacji
 - parametry: **trip_id**, **person_id**
 - procedura powinna kontrolować czy wycieczka jeszcze się nie odbyła, i czy są wolne miejsca
 - procedura powinna również dopisywać inf. do tabeli **log**
- **p_modify_reservation_status**
 - zadaniem procedury jest zmiana statusu rezerwacji
 - parametry: **reservation_id**, **status**
 - dopuszczalne są wszystkie zmiany statusu
 - ale procedura powinna kontrolować czy taka zmiana jest możliwa, np. zmiana statusu już anulowanej wycieczki (przywrócenie do stanu aktywnego nie zawsze jest możliwa – może już nie być miejsc)
 - procedura powinna również dopisywać inf. do tabeli **log**
- **p_modify_max_no_places**
 - zadaniem procedury jest zmiana maksymalnej liczby miejsc na daną wycieczkę
 - parametry: **trip_id**, **max_no_places**
 - nie wszystkie zmiany liczby miejsc są dozwolone, nie można zmniejszyć liczby miejsc na wartość poniżej liczby zarezerwowanych miejsc

Należy rozważyć użycie transakcji

- czy należy użyć **commit** wewnątrz procedury w celu zatwierdzenia transakcji
 - jakie są tego konsekwencje

Należy zwrócić uwagę na kontrolę parametrów (np. jeśli parametrem jest **trip_id** to należy sprawdzić czy taka wycieczka istnieje, jeśli robimy rezerwację to należy sprawdzać czy są wolne miejsca itp..)

Proponowany zestaw procedur można rozbudować wedle uznania/potrzeb

- np. można dodać nowe/pomocnicze funkcje/procedury

Zadanie 3 - rozwiązanie

```

-- wyniki, kod, zrzuty ekranów, komentarz ...
create procedure p_add_reservation(vtrip_id number, vperson_id number)
as
    validtrip          number;
    availabletrip      number;
    validperson        number;

```

```

    new_reservation_id reservation.reservation_id%type;
begin
    select count(*)
    into validtrip
    from trip t
    where t.trip_id = vtrip_id;

    if validtrip = 0 then
        raise_application_error(-20001, 'trip not found');
    end if;

    select count(*)
    into validperson
    from person p
    where p.person_id = vperson_id;

    if validperson = 0 then
        raise_application_error(-20001, 'person not found');
    end if;

    select count(*)
    into availabletrip
    from vw_available_trip vw
    where vw.trip_id = vtrip_id;

    if availabletrip = 0 then
        raise_application_error(-20001, 'trip not available');
    end if;

    insert into reservation(trip_id, person_id, status)
    values (vtrip_id, vperson_id, 'N')
    returning reservation_id into new_reservation_id;

    insert into log(reservation_id, log_date, status)
    values (new_reservation_id, sysdate, 'N');

end;
/

create procedure p_modify_reservation_status(vreservation_id number, vstatus char)
as
    valid_reservation number;
    available_trip     number;
begin
    select count(*)
    into valid_reservation
    from reservation
    where vreservation_id = reservation_id;

    if valid_reservation = 0 then
        raise_application_error(-20001, 'reservation not found');
    end if;

    if vstatus != 'C' then
        select count(*)
        into available_trip
        from vw_available_trip vw_a
        inner join reservation r on vw_a.trip_id = r.trip_id
        where vreservation_id = r.reservation_id;

        if available_trip = 0 then
            raise_application_error(-20001, 'trip not available');
        end if;
    end if;

    update reservation set status = vstatus where vreservation_id = reservation_id;

    insert into log(reservation_id, log_date, status)
    values (vreservation_id, sysdate, vstatus);
end;

create procedure p_modify_max_no_places(vtrip_id number, vmax_no_places number)
as
    valid_trip number;
    available_trip number;
begin
    select count(*)
    into valid_trip

```



```

from trip
where vtrip_id = trip_id;

if valid_trip = 0 then
    raise_application_error(-20001, 'trip not found');
end if;

select places_left + (vmax_no_places - max_no_places)
into available_trip
from vw_available_trip
where vtrip_id = trip_id;

if available_trip < 0 then
    raise_application_error(-20001, 'cannot lower the max number of places');
end if;

update trip set max_no_places = vmax_no_places where vtrip_id = trip_id;
end;
/

```

Zadanie 4 - triggerzy

Zmiana strategii zapisywania do dziennika rezerwacji. Realizacja przy pomocy triggerów

Należy wprowadzić zmianę, która spowoduje, że zapis do dziennika będzie realizowany przy pomocy triggerów

Triggerzy:

- trigger/triggery obsługujące
 - dodanie rezerwacji
 - zmianę statusu
- trigger zabraniający usunięcia rezerwacji

Oczywiście po wprowadzeniu tej zmiany należy "uaktualnić" procedury modyfikujące dane.

UWAGA Należy stworzyć nowe wersje tych procedur (dodając do nazwy dopisek 4 - od numeru zadania). Poprzednie wersje procedur należy pozostawić w celu umożliwienia weryfikacji ich poprawności

Należy przygotować procedury: `p_add_reservation_4`, `p_modify_reservation_status_4`, `p_modify_reservation_4`

Zadanie 4 - rozwiązanie

```

create procedure p_add_reservation_4(vtrip_id number, vperson_id number)
as
    validtrip      number;
    availabletrip  number;
    validperson    number;
    new_reservation_id reservation.reservation_id%type;
begin
    select count(*)
    into validtrip
    from trip t
    where t.trip_id = vtrip_id;

    if validtrip = 0 then
        raise_application_error(-20001, 'trip not found');
    end if;

    select count(*)
    into validperson
    from person p
    where p.person_id = vperson_id;

    if validperson = 0 then
        raise_application_error(-20001, 'person not found');
    end if;

    select count(*)
    into availabletrip
    from vw_available_trip vw
    where vw.trip_id = vtrip_id;

    if availabletrip = 0 then
        raise_application_error(-20001, 'trip not available');
    end if;

    insert into reservation(trip_id, person_id, status)

```

```

        values (vtrip_id, vperson_id, 'N')
        returning reservation_id into new_reservation_id;

end;
/

create or replace trigger tr_add_reservation
after insert
on reservation
for each row
begin
    insert into log(reservation_id, log_date, status)
    values (:NEW.reservation_id, sysdate, 'N');
end;

create procedure p_modify_reservation_status_4(vreservation_id number, vstatus char)
as
    valid_reservation number;
    available_trip    number;
begin
    select count(*)
    into valid_reservation
    from reservation
    where vreservation_id = reservation_id;

    if valid_reservation = 0 then
        raise_application_error(-20001, 'reservation not found');
    end if;

    if vstatus != 'C' then
        select count(*)
        into available_trip
        from vw_available_trip vw_a
        inner join reservation r on vw_a.trip_id = r.trip_id
        where vreservation_id = r.reservation_id;

        if available_trip = 0 then
            raise_application_error(-20001, 'trip not available');
        end if;
    end if;

    update reservation set status = vstatus where vreservation_id = reservation_id;
end;
/

create or replace trigger tr_modify_reservation_status
after update on reservation for each row
begin
    insert into log(reservation_id, log_date, status)
    values (:NEW.reservation_id, sysdate, :NEW.status);
end;

create or replace trigger delete_reservation_block
before delete on reservation for each row
begin
    RAISE_APPLICATION_ERROR(-20001, 'Nie mozna usuwac rezerwacji!');
end;

```

Zadanie 5 - triggerzy

Zmiana strategii kontroli dostępności miejsc. Realizacja przy pomocy triggerów

Należy wprowadzić zmianę, która spowoduje, że kontrola dostępności miejsc na wycieczki (przy dodawaniu nowej rezerwacji, zmianie statusu) będzie realizowana przy pomocy triggerów

Triggerzy:

- Trigger/triggery obsługujące:
 - dodanie rezerwacji
 - zmianę statusu

Oczywiście po wprowadzeniu tej zmiany należy "uaktualnić" procedury modyfikujące dane.

UWAGA Należy stworzyć nowe wersje tych procedur (np. dodając do nazwy dopisek 5 - od numeru zadania). Poprzednie wersje procedur należy pozostawić w celu umożliwienia weryfikacji ich poprawności.

Należy przygotować procedury: `p_add_reservation_5`, `p_modify_reservation_status_5`, ...

Zadanie 5 - rozwiązanie

```
create procedure p_add_reservation_5(vtrip_id number, vperson_id number)
as
    validtrip          number;
    validperson        number;
    new_reservation_id reservation.reservation_id%type;
begin
    select count(*)
    into validtrip
    from trip t
    where t.trip_id = vtrip_id;

    if validtrip = 0 then
        raise_application_error(-20001, 'trip not found');
    end if;

    select count(*)
    into validperson
    from person p
    where p.person_id = vperson_id;

    if validperson = 0 then
        raise_application_error(-20001, 'person not found');
    end if;

    insert into reservation(trip_id, person_id, status)
    values (vtrip_id, vperson_id, 'N')
    returning reservation_id into new_reservation_id;

    insert into log(reservation_id, log_date, status)
    values (new_reservation_id, sysdate, 'N');

end;
/

create trigger tr_check_avaiable
before insert
on reservation
for each row
declare
    availabletrip number;
begin
    select count(*)
    into availabletrip
    from vw_available_trip vw
    where vw.trip_id = :NEW.trip_id;

    if availabletrip = 0 then
        raise_application_error(-20001, 'trip not avaiable');
    end if;
end;
/

create or replace trigger tr_modify_reservation_status_avaiable
before update
on reservation
for each row
declare
    available_trip number;
begin
    if :NEW.status != 'C' then
        select count(*)
        into available_trip
        from vw_available_trip vw_a
             inner join reservation r on vw_a.trip_id = r.trip_id
        where :NEW.reservation_id = r.reservation_id;

        if available_trip = 0 then
            raise_application_error(-20001, 'trip not avaiable');
        end if;
    end if;
end;
end;
```

Zadanie - podsumowanie

Porównaj sposób programowania w systemie Oracle PL/SQL ze znanym ci systemem/językiem MS Sqlserver T-SQL

Podsumowanie – Oracle PL/SQL vs MS SQL Server T-SQL

1. Podejście do programowania

- **PL/SQL (Oracle)** – rozbudowany język programowania w bazie (logika biznesowa, typy, kolekcje)
- **T-SQL (SQL Server)** – prostszy, bardziej skupiony na operacjach na danych

2. Transakcje

- **Oracle** – transakcje automatyczne, kończone przez **COMMIT** / **ROLLBACK**, DDL = automatyczny commit
- **T-SQL** – transakcje jawne (**BEGIN TRAN**), DDL można cofnąć

3. Obsługa błędów

- **Oracle** – **raise_application_error**, **EXCEPTION**
- **T-SQL** – **TRY...CATCH**

4. Funkcje i typy

- **Oracle** – własne typy i kolekcje (zwracanie tabel)
- **T-SQL** – funkcje tabelaryczne, mniej elastyczne

5. Procedury

- **Oracle** – bardziej rozbudowane, zawierają logikę i walidację
- **T-SQL** – prostsze, głównie operacje na danych

6. Triggery

- **Oracle** – poziom rekordu (:NEW, :OLD)
- **T-SQL** – poziom zbioru (**INSERTED**, **DELETED**)

7. Klucze główne

- **Oracle** – **SEQUENCE**
- **T-SQL** – **IDENTITY**

Wniosek

- **Oracle PL/SQL** – lepszy do złożonej logiki w bazie
- **T-SQL** – prostszy i czytelniejszy do pracy z danymi